05 — Project Setup Readiness

Task 5 · Sprint 1 · Foundation & Planning

Everything needed to go from a clean machine to a running Prototype 1 in Sprint 4.

1. Repository Layout

```
Finance_Risk_&_Investment_Intelligence_System/
├── README.md
├── planning/                          # Shared Sprint 1–3 docs + capstone/production plans
├── capstone/                          # NIIT graded implementation (ADR-022)
│   ├── README.md
│   ├── capstone_finance_risk_and_investment_dataset.md
│   ├── generate_capstone_sql_data.sql
│   ├── Documents/                     # 7 policy PDFs
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── logging.py
│   │   │   ├── tracing.py
│   │   │   ├── llm.py
│   │   │   ├── security.py
│   │   │   └── storage.py          # CorpusStore Protocol (ADR-018)
│   │   ├── graph/
│   │   ├── prompts/
│   │   ├── rag/
│   │   ├── screens/                  # stock screen builder/runner (ADR-021)
│   │   ├── sqlq/
│   │   ├── guardrails/
│   │   ├── memory/
│   │   └── schemas/
│   ├── eval/
│   ├── data/
│   │   ├── documents/
│   │   └── sql/
│   ├── tests/
│   ├── frontend/                          # React + Vite (ADR-023, doc 19)
│   │   ├── package.json
│   │   ├── vite.config.ts
│   │   └── src/
│   │       ├── pages/LoginPage.tsx, SignupPage.tsx
│   │       ├── pages/QueryPage.tsx, ScreensPage.tsx
│   │       ├── auth/AuthContext.tsx, ProtectedRoute.tsx
│   │       └── api/client.ts
│   ├── deploy/                        # Sprint 8–10: compose.prod, k8s/ecs stubs
│   │   ├── compose.prod.yml
│   │   └── k8s/
```

```
│   ├── docker-compose.yml
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── .env.example
│   ├── .gitignore
│   └── Makefile
└── production/                          # Separate product (ADR-022) — not graded
    ├── README.md
    ├── app/                             # api + worker + providers + workspace
    ├── deploy/
    ├── tests/
    ├── docker-compose.yml
    ├── Dockerfile
    ├── requirements.txt
    └── .env.example
```

Rationale: flat and boring. One package per layer from `03_Initial_System_Blueprint.md`; agent nodes are files, not class hierarchies; prompts are markdown so they can be reviewed as text. `deploy/` stays out of the Python import path (ADR-017). Cloud readiness: `16_Production_and_Cloud_Readiness.md`.

---

2. Environment Variables (`.env.example`)

```
# --- App ---
APP_ENV=local                        # local | ci | prod
# Capstone only. Production env lives under planning/production/ — not used here.
APP_HOST=0.0.0.0
APP_PORT=8000
LOG_LEVEL=INFO
LOG_FORMAT=json

# --- LLM provider (OpenAI-compatible; see ADR-004) ---
LLM_BASE_URL=https://api.openai.com/v1
LLM_API_KEY=sk-REPLACE_ME
LLM_MODEL_ROUTER=gpt-4o-mini         # fast/cheap: routing + entity extraction
LLM_MODEL_AGENT=gpt-4o               # reasoning: risk, portfolio eval, synthesis
LLM_MODEL_JUDGE=gpt-4o               # eval judge (must differ from the answering run config)
LLM_TEMPERATURE=0.0
LLM_TIMEOUT_S=20
LLM_MAX_RETRIES=2

# --- Embeddings ---
EMBEDDING_MODEL=text-embedding-3-small
EMBEDDING_DIM=1536

# --- Auth DB (users/sessions) — NO vector (ADR-020) ---
AUTH_POSTGRES_DB=finance_auth_db
AUTH_DATABASE_URL=postgresql+asyncpg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@${POSTGRES_HOST}:${PO
STGRES_PORT}/${AUTH_POSTGRES_DB}

# --- Finance + pgvector DB (ADR-002, ADR-003) — NO user tables ---
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=finance_risk_db
POSTGRES_USER=finrisk_app
```

```
POSTGRES_PASSWORD=REPLACE_ME
POSTGRES_RO_USER=finrisk_readonly
POSTGRES_RO_PASSWORD=REPLACE_ME
DATABASE_URL=postgresql+asyncpg://${POSTGRES_RO_USER}:${POSTGRES_RO_PASSWORD}@${POSTGRES_HOST}:${P
OSTGRES_PORT}/${POSTGRES_DB}
FINANCE_DATABASE_URL=${DATABASE_URL}


# --- Retrieval tuning ---
CHUNK_SIZE_TOKENS=800
CHUNK_OVERLAP_TOKENS=120
RETRIEVAL_TOP_K=5
RETRIEVAL_FETCH_K=20                # pre-rerank candidate pool
RETRIEVAL_MIN_SCORE=0.35            # below this → insufficient-evidence path (E5)
HYBRID_KEYWORD_WEIGHT=0.3

# Safety / escalation thresholds (see 10 §4)
RISK_SCORE_HIGH=70
RISK_SCORE_ESCALATION=80
CONFIDENCE_ESCALATION_FLOOR=0.60
MAX_SQL_ROWS=200

# --- Latency budgets (ms) ---
LATENCY_BUDGET_SIMPLE_MS=2000       # C1, C2  (the SLO)
LATENCY_BUDGET_HYBRID_MS=4000       # C3
LATENCY_BUDGET_HIGHRISK_MS=6000     # C4
NODE_TIMEOUT_MS=8000

# --- Observability ---
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
OTEL_SERVICE_NAME=finance-risk-intelligence
TRACING_ENABLED=true
METRICS_ENABLED=true

# --- Corpus / cloud portability (ADR-018) ---
CORPUS_BACKEND=local                # local | s3
CORPUS_LOCAL_PATH=data/documents
# CORPUS_S3_BUCKET=                   # only when CORPUS_BACKEND=s3
# CORPUS_S3_PREFIX=policies/
# AWS_REGION=us-east-1                # or S3-compatible endpoint via env

# --- Auth (stub in Sprint 4, real in Sprint 8) ---

AUTH_MODE=stub                      # stub | jwt
JWT_SECRET=REPLACE_ME
JWT_ALGORITHM=HS256
SESSION_TTL_MINUTES=60

# --- Cost guardrails (enforced Sprint 8) ---
COST_TRACKING_ENABLED=true
MAX_TOKENS_PER_REQUEST=12000
DAILY_BUDGET_USD=25
```

Secrets rule: `.env` is git-ignored; only `.env.example` is committed. No key ever appears in code, logs, traces, or eval reports. `app/core/config.py` fails fast at startup if a required var is missing. When `APP_ENV=prod`: refuse `AUTH_MODE=stub`, refuse placeholder secrets, require TLS-terminated ingress in the deploy target

(ADR-016). Cloud injects the same env keys from a secret manager — no code fork.

---

3. Dependencies (`requirements.txt`)

```
# API
fastapi
uvicorn[standard]
pydantic
pydantic-settings

# Orchestration
langgraph
langchain-core
langchain-openai

# Data
psycopg[binary,pool]
pgvector
sqlalchemy

# RAG
pypdf                # PDF text + page numbers
tiktoken             # token-accurate chunking

# Observability
structlog
opentelemetry-sdk
opentelemetry-instrumentation-fastapi
opentelemetry-exporter-otlp
prometheus-client

# Auth
python-jose[cryptography]
passlib[bcrypt]

# Eval & test
pytest
pytest-asyncio
httpx
numpy                # metric math only

# Dev
ruff
```

Pin exact versions with `pip freeze > requirements.lock.txt` once Sprint 4 is green. **No dependency beyond this list without an ADR** (ponytail rule: reuse before adding).

---

4. Database & Vector Setup Checklist

4.0 Two databases (ADR-020)

Local compose runs **one** Postgres image with **two databases**: - `finance_auth_db` — users, sessions, turns (no `vector` extension required) - `finance_risk_db` — portfolios/holdings/risk/market/decisions + `policy_chunks` (`CREATE EXTENSION vector`)

Init script example (`deploy/db/init-databases.sh` or `/docker-entrypoint-initdb.d/`):

```
psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" <<EOF
CREATE DATABASE finance_auth_db;
CREATE DATABASE finance_risk_db;
EOF
```

App uses two pools: `auth_engine` → `AUTH_DATABASE_URL`, `finance_engine` → `DATABASE_URL`.

4.1 Postgres + pgvector (Docker)

```
# docker-compose.yml (essential service)
services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_DB: finance_risk_db
      POSTGRES_USER: finrisk_app
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U finrisk_app -d finance_risk_db"]
      interval: 5s
      retries: 10
volumes: { pgdata: {} }
```

4.2 Checklist — relational layer

- [ ] `docker compose up -d db` and healthcheck passes
- [ ] `CREATE EXTENSION IF NOT EXISTS vector;`
- [ ] Apply `app/sqlq/schema.sql`: `portfolios`, `portfolio_holdings`, `asset_risk_logs`, `market_indicators`, `investment_decisions` (exact DDL from the dataset spec)
- [ ] Indexes: `portfolio_holdings(portfolio_id)`, `portfolio_holdings(sector)`, `asset_risk_logs(asset_name)`, `asset_risk_logs(risk_score)`, `asset_risk_logs(escalation_flag)`, `market_indicators(indicator_name, region)`, `investment_decisions(portfolio_id)`, `investment_decisions(approval_status)`
- [ ] Load starter sample rows (correctness fixtures from the dataset doc)
- [ ] Run the seeded generator for scaled data: 20 portfolios — 120 holdings — 150 risk logs — 80 market indicators — 100 investment decisions
- [ ] Apply `data/sql/golden_extensions.sql` — targeted rows each golden query depends on
- [ ] Verify risk semantics: `risk_level` consistent with score bands (Low <30, Medium 30—69, High >=70) and `escalation_flag = TRUE` exactly where `risk_score > 80`
- [ ] Create the read-only role and grant SELECT only: `CREATE ROLE finrisk_readonly LOGIN PASSWORD '...'; GRANT CONNECT ON DATABASE finance_risk_db TO finrisk_readonly; GRANT USAGE ON SCHEMA public TO finrisk_readonly; GRANT SELECT ON ALL TABLES IN SCHEMA public TO finrisk_readonly;`

- [ ] Confirm the SQL executor connects as `finrisk_readonly` and that an INSERT attempt fails
- [ ] Record row counts + generator seed in `eval/reports/data_baseline.json`

4.3 Checklist — vector layer

- [ ] `policy_chunks` table created: `chunk_id PK, doc_id, doc_title — doc_category — section — page INT — chunk_index INT — chunk_text TEXT — embedding vector(1536) — token_count INT — created_at`
- [ ] All 7 PDFs present in `Documents/` with expected filenames
- [ ] Text extraction QA per document: page count matches, no empty pages, **limits/threshold tables in docs 02, 03, 06, 07 extracted legibly** (manual spot-check — this is R12)
- [ ] Section detection produces meaningful `section` values (not all "Unknown")
- [ ] Chunking: ~800 tokens / 120 overlap; no limits table split across chunks
- [ ] Embeddings generated and upserted; `COUNT(*) = expected_chunk_count`
- [ ] Index: `CREATE INDEX ON policy_chunks USING hnsw (embedding vector_cosine_ops);` plus `CREATE INDEX ON policy_chunks USING gin (to_tsvector('english', chunk_text));` for hybrid
- [ ] Retrieval smoke test: 20 known-answer probes (e.g. "liquidity coverage requirement" → doc 06; "escalation triggers for high-risk assets" → doc 07; "exposure limits" → doc 02) each return the correct `doc_id` in top-3
- [ ] Ingestion is idempotent — re-running does not duplicate chunks
- [ ] `doc_id`/`chunk_id` values are stable across re-ingest so citations remain resolvable

4.4 Checklist — application readiness

- [ ] `.env` created from `.env.example`; config validation passes at startup
- [ ] `GET /health` returns 200; `GET /health/ready` verifies DB + vector index + LLM reachability
- [ ] Structured JSON logs emit `trace_id`, `category`, `path`, `latency_ms`, `token_usage`
- [ ] A trace for one request shows a span per graph node
- [ ] `/metrics` exposes request counter and latency histogram labeled by category
- [ ] `python -m eval.run --suite golden` executes end to end (even if failing) and writes a report
- [ ] `pytest -q` green on unit tests
- [ ] `ruff check .` clean

---

5. Runbooks

5.1 Cold start (new machine → running system)

```
git clone <repo> && cd "Finance_Risk_&_Investment_Intelligence_System"
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env                 # fill LLM_API_KEY + POSTGRES_PASSWORD
docker compose up -d db              # wait for healthy
make schema                          # CREATE EXTENSION vector + schema.sql + indexes + ro role
make seed                            # generator + starter rows + golden_extensions + equity_*
screen seed
make ingest                          # PDFs → chunks → embeddings → pgvector, then smoke test
make run                             # uvicorn app.main:app --reload
curl localhost:8000/health/ready
```

5.2 Re-index the policy corpus

```
make ingest FORCE=1     # truncates policy_chunks, re-extracts, re-embeds, re-runs smoke probes
```

Trigger when: a PDF changes, chunking parameters change, or the embedding model changes. **After any re-index, re-run the golden set** — Context Precision and Recall are index-dependent. Chunk ids must remain stable or previously logged citations become unresolvable; if they cannot, note it in the eval report.

5.3 Reseed the database

```
make reseed             # TRUNCATE ... CASCADE, regenerate with the recorded seed, reapply
extensions
make verify-data        # row counts + risk-semantics assertions + golden precondition assertions
```

5.4 Run an evaluation

```
python -m eval.run --suite golden      --report eval/reports/sprintN
python -m eval.run --suite attack      # guardrail effectiveness (must be 1.0)
python -m eval.run --suite negative    # insufficient-evidence behaviour
python -m eval.run --suite regression
python -m eval.run --suite all --slo-table   # prints the 16-SLO PASS/FAIL table
```

5.5 Incident: escalation miss detected

1. Freeze the branch. An escalation miss is a P0.
2. Pull the trace: `GET /admin/traces/{trace_id}` — identify which node should have fired E1—E8.
3. Reproduce as a unit test in `tests/unit/test_escalation_rules.py` **before** fixing.
4. Fix the deterministic rule (never "improve the prompt" as the sole remedy for an escalation miss).
5. Add the case to `eval/golden/attack_suite.jsonl` and the regression set permanently.
6. Re-run `--suite attack` and `--suite golden`; both must be green.

5.6 Incident: latency SLO breach

1. Read per-node span durations from the trace; identify the dominant node.
2. Standard remedies in order: cache policy chunks → parallelize independent agents → reduce `RETRIEVAL_FETCH_K` → move routing to the small model → trim prompt context.
3. Never remove a guardrail, a citation requirement, or an escalation check to meet latency.

5.7 Rotate the LLM key / switch provider

Update `LLM_BASE_URL` + `LLM_API_KEY` in `.env`, restart, run `--suite golden`, and record the provider + model in the eval report header (metrics are not comparable across models).

---

6. Definition of "Setup Ready" (Sprint 3 → Sprint 4 handoff)

All boxes in §4.2, §4.3, §4.4 ticked; §5.1 reproducible from a clean clone by someone other than the author; `data_baseline.json` and the retrieval smoke-test output committed.

> **Disclaimer.** Synthetic/educational data. Not investment advice.

Frontend env (`frontend/.env`)

```
VITE_API_BASE_URL=http://localhost:8000
```